# Memoria PAI-2 BYODSEC

Oct 26, 2026 · @Fran

Grupo 20 — Security Team. Asignatura: Seguridad en Sistemas Informáticos e Internet (SSII).

## 1. Introducción

El proyecto implementa un gateway de acceso seguro tipo "road warrior" para dispositivos BYOD: un canal cifrado por el que los usuarios de la universidad (PDI, PAS e investigadores) se dan de alta, se autentican y envían mensajes protegidos desde fuera de la red corporativa.

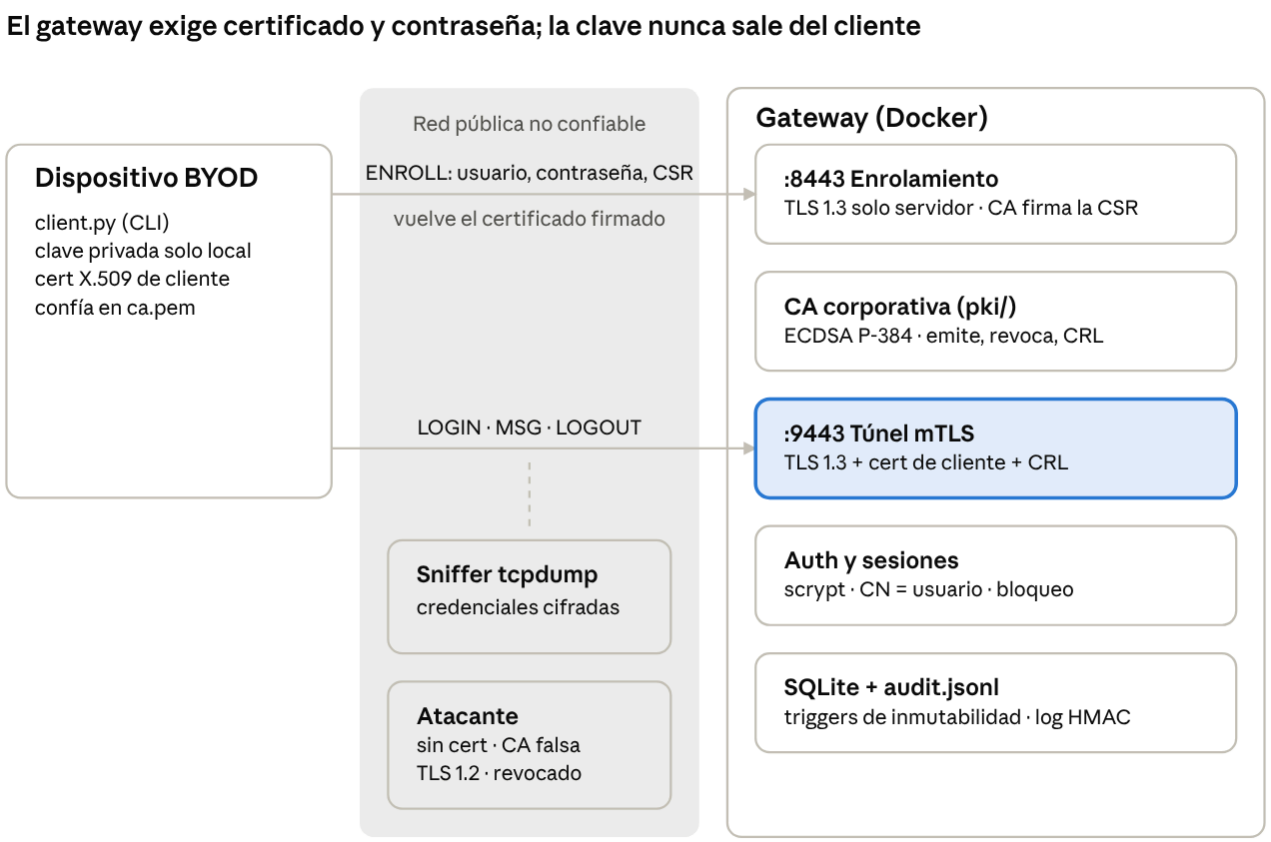
El sistema se apoya en tres pilares: una infraestructura de clave pública (PKI) propia que emite y revoca certificados, un canal TLS 1.3 con autenticación mutua (mTLS) que garantiza confidencialidad, integridad y secreto hacia adelante, y una capa de autenticación por usuario y contraseña con registro de auditoría íntegro.

| Dato | Valor |
|---|---|
| Equipo | Grupo 20 — Security Team |
| Lenguaje y entorno | Python ≥ 3.11, `asyncio`, SQLite, Docker Compose |
| Puertos | 8443 (enrolamiento), 9443 (canal seguro mTLS) |
| Usuarios precargados | 300 (120 PDI, 100 PAS, 80 INV) |

El documento se organiza en tres apartados según el enunciado: un **resumen** de las decisiones técnicas y algorítmicas, un **manual de despliegue y uso**, y el **grado de completitud** frente a los objetivos y requisitos. La numeración de requisitos funcionales sigue la del enunciado, que salta de RF2 a RF4.

## 2. Resumen de decisiones técnicas y algorítmicas

El servidor es un único proceso `asyncio` que atiende dos puertos: el 8443 para el alta de dispositivos (enrolamiento) y el 9443 para el canal seguro. Una corrutina por conexión permite sostener cientos de sesiones simultáneas sin hilos por cliente.




arquitectura del sistema · enrolamiento (8443) y canal seguro mTLS (9443)

El puerto 8443 solo sirve para dar de alta un dispositivo; todo lo demás pasa por el 9443, que rechaza el handshake si no hay un certificado válido y no revocado.

## 2.1 PKI e identidad

La autoridad de certificación (CA) raíz es **ECDSA sobre la curva P-384 con firma SHA-384**, autofirmada, con `BasicConstraints ca=True` y `KeyUsage` de firma de certificados y de CRL. Emite certificados X.509v3 tanto para el servidor como para los clientes, incluyendo las extensiones `KeyUsage`, `ExtendedKeyUsage` (`serverAuth` / `clientAuth`), `SubjectKeyIdentifier` y `AuthorityKeyIdentifier`, y un `SubjectAlternativeName` en el del servidor. Se elige curva elíptica frente a RSA por tamaño de clave y coste de handshake menores a igual nivel de seguridad.

En el **alta (RF1)**, la clave privada del cliente se genera y permanece **siempre en el dispositivo**: el cliente crea su par de claves y una CSR con `CN = usuario`, y envía solo la CSR junto a sus credenciales. La CA la firma únicamente si las credenciales son válidas y el CN coincide con el usuario autenticado. La revocación mantiene una **CRL firmada** por la CA que el gateway consulta en cada handshake.

## 2.2 Canal seguro: TLS 1.3 + mTLS

El canal fija **TLS 1.3 como versión mínima y máxima**, rechazando cualquier intento de versión anterior. Las suites se restringen a dos AEAD de 256 bits, `TLS_AES_256_GCM_SHA384` y `TLS_CHACHA20_POLY1305_SHA256`. Como Python no expone la fijación de suites de TLS 1.3 desde la API de `ssl`, se cargan mediante un `openssl.cnf` propio a través de `OPENSSL_CONF`, que debe estar definido antes de importar `ssl`; una comprobación en el arranque aborta el servicio si OpenSSL ofreciera cualquier otra suite. El secreto hacia adelante (PFS) lo aporta el intercambio efímero (EC)DHE de TLS 1.3, reforzado desactivando los tickets de sesión (`OP_NO_TICKET`).

La autenticación mutua exige certificado de cliente (`CERT_REQUIRED`) validado contra la CA y la CRL. Así, un cliente sin certificado, con un certificado de otra CA o con uno revocado no completa el handshake.

## 2.3 Autenticación en dos capas y almacenamiento

El **login (RF2)** combina dos factores independientes: el certificado presentado en el handshake mTLS y la pareja usuario/contraseña. Además, el `CN` del certificado debe coincidir con el usuario que inicia sesión, de modo que un certificado válido no sirve para suplantar a otro usuario.

Las contraseñas se almacenan con **`scrypt`** (parámetros N=16384, r=8, p=1, clave derivada de 64 bytes) y una sal aleatoria por usuario; la verificación se hace en tiempo constante. Tras **5 intentos fallidos** la cuenta se bloquea durante 15 minutos. Los datos viven en SQLite, en las tablas `users` y `certificates`.

## 2.4 Inmutabilidad y auditoría

La integridad de la identidad se protege con **triggers de inmutabilidad** en SQLite: cualquier `UPDATE` sobre los campos de identidad o criptográficos de un certificado (usuario, clave pública, huella, número de serie) y cualquier `DELETE` sobre la tabla de certificados se abortan; solo se permite pasar un certificado a estado revocado.

La **auditoría (RS4)** registra cada evento relevante (alta, conexión, fallo de handshake, login correcto o fallido, bloqueo, cierre) en un fichero JSON Lines donde cada línea encadena un **HMAC-SHA256** con el hash de la anterior. Alterar o borrar una línea rompe la cadena, lo que detecta el verificador `verify_audit.py`.

# 3. Manual de despliegue y uso

## 3.1 Requisitos

Python 3.11 o superior y las dependencias de `requirements.txt` (`cryptography`, `psutil`, `matplotlib`, `pytest`). Para capturar tráfico `.pcap` se necesita `tcpdump` (Linux o WSL). Todo lo demás es de la biblioteca estándar.

```
pip install -r requirements.txt
```

## 3.2 Preparación de la PKI y los usuarios

Desde la carpeta `PAI2/`, se genera la CA, el certificado de servidor, la CRL y los 300 usuarios precargados (RF5):

```
python -c "from pki.ca import generate_ca; generate_ca(force=True)"
python -c "from pki.issue import issue_server_cert; issue_server_cert()"
python -c "from pki.crl import generate_crl; generate_crl([])"
python -m seed.seed_users
```

Las claves privadas no se versionan; cada despliegue regenera su propia PKI con estos scripts.

## 3.3 Arranque del gateway

OpenSSL debe cargar la política de suites antes de que Python importe `ssl`, por lo que se exporta `OPENSSL_CONF` (el propio `gateway.py` también la fija):

```
export OPENSSL_CONF=$PWD/server/openssl.cnf     # en PowerShell:
$env:OPENSSL_CONF="$PWD\server\openssl.cnf"
python server/gateway.py
```

El servidor queda escuchando en el 8443 (enrolamiento) y el 9443 (canal seguro).

## 3.4 Uso: alta y sesión segura

Un dispositivo nuevo se da de alta generando su clave y CSR en local y enviándolas con sus credenciales:

```
python client/enroll_client.py <usuario> <contraseña>
```

Una vez con certificado, el cliente abre el canal mTLS y realiza el flujo completo (login en dos capas, mensaje seguro y cierre):

```
python client/client.py
```

Las operaciones del protocolo (tramas de 4 bytes de longitud + JSON) son:

| Operación | Puerto | Descripción |
|---|---|---|
| `ENROLL` | 8443 | Alta: valida credenciales, firma la CSR y devuelve el certificado |
| `LOGIN` | 9443 | Autenticación en dos capas (mTLS + usuario/contraseña) |
| `MSG` | 9443 | Envío de mensaje sobre el canal cifrado |
| `LOGOUT` | 9443 | Cierre de sesión y del canal con `close_notify` |

### 3.5 Despliegue con Docker

El entorno reproducible levanta el gateway, un sniffer que comparte su red, el cliente de carga y el módulo de ataques:

```
cd docker
docker compose run --rm ca-init           # genera PKI y usuarios
docker compose up -d gateway sniffer
docker compose run --rm bench             # prueba de carga
docker compose run --rm attacker          # batería de ataques
```

### 3.6 Verificación

```
pytest -q                # 16 pruebas (PKI, auth, inmutabilidad, auditoría,
ataques)
python -m server.verify_audit   # comprueba la cadena HMAC de la auditoría
```

## 4. Grado de completitud

Todos los objetivos y requisitos del enunciado están implementados y verificados con código y evidencias. Las pruebas automáticas pasan al completo (**16/16**) y las medidas y capturas se encuentran en `evidence/`.

## 4.1 Matriz de trazabilidad

| Requisito | Módulo | Verificación / evidencia |
|---|---|---|
| Obj.1 · Canal seguro para credenciales y mensajes | `server/gateway.py`, `client/client.py` | Flujo completo; `test_protocol`, captura `02_tunnel_login.pcap` |
| Obj.2 · PKI + TLS 1.3 + mTLS | `pki/*`, `server/tls.py` | `test_pki`, handshake TLS 1.3 con suite AES-256-GCM |
| Obj.3 · Rendimiento 300 usuarios | `bench/*` | 300/300 sesiones; escalado hasta 500 |
| Obj.4 · Auditoría de tráfico | `attacks/*`, `server/audit.py` | capturas `.pcap` y `audit.jsonl` |
| Obj.5 · Suplantación / MitM (extra) | `attacks/*` | 7/7 ataques rechazados |
| RF1 · Alta con certificado e inmutabilidad | `server/enroll.py`, triggers en `db.py` | `test_enroll`, `test_immutability` |
| RF2 · Login en dos capas | `server/gateway.py`, `server/auth.py` | `test_auth`, eventos `LOGIN_OK` / `LOGIN_FAIL` |
| RF4 · Cierre de sesión | `server/gateway.py` | `LOGOUT` con `close_notify` |
| RF5 · 300 usuarios precargados | `seed/seed_users.py` | 300 certificados emitidos |
| RS1 · CA y mTLS estricto | `pki/*`, `server/tls.py` | rechazo sin cert / revocado |
| RS2 · TLS 1.3 y suites AEAD con PFS | `server/tls.py`, `server/openssl.cnf` | rechazo de TLS 1.2 y de suite de 128 bits |
| RS3 · 300 sesiones + comparativa | `bench/*` | gráficas TLS vs claro |
| RS4 · `.pcap` + registros íntegros | `docker/capture.sh`, `server/audit.py` | `evidence/pcap/*`, cadena HMAC verificada |

## 4.2 Rendimiento (RS3)

Se compara el gateway TLS 1.3 + mTLS con un servidor de referencia en claro que ejecuta la misma lógica. El sistema sostiene **300 y 500 sesiones concurrentes sin fallos**.

| Métrica (300 sesiones) | TCP en claro | TLS 1.3 + mTLS |
|---|---|---|
| Sesiones correctas | 300 / 300 | 300 / 300 |
| Conexión + handshake (p50) | 18 ms | 743 ms |
| RTT por mensaje (p50) | 11,9 ms | 13,2 ms |
| Throughput de mensajes | — | ≈ 22 400 msg/s |
| Memoria máxima del servidor | 133 MiB | 230 MiB |

El coste de TLS se concentra en el handshake (criptografía asimétrica P-384); una vez abierto el canal, cada mensaje cuesta solo un 11 % más que en claro. Las gráficas del escalado 50→500 están en `evidence/charts/`.

## 4.3 Batería de ataques (Obj.5, extra)

Cada ataque es un intento de acceso que **debe ser rechazado**. Resultado: **7/7 rechazados**, cada uno con su captura o traza de auditoría.

| Ataque | Resultado |
|---|---|
| Sniffing pasivo de credenciales | Ilegible en el túnel (visible solo en claro) |
| Cliente sin certificado | Handshake abortado |
| Certificado de CA falsa | Rechazado (cadena no confiable) |
| Suplantación de servidor con CA falsa | El cliente aborta antes de enviar credenciales |
| Downgrade a TLS 1.2 | Rechazado (min = max = TLS 1.3) |
| Suite débil de 128 bits | Rechazado ( `NO_SHARED_CIPHER` ) |
| Certificado revocado | Rechazado por la CRL |
| Fuerza bruta de contraseña | Cuenta bloqueada a los 5 intentos |

La integridad del canal la garantiza el cifrado AEAD de TLS 1.3: cualquier bit alterado en un registro rompe su etiqueta de autenticación y el registro se descarta.

## 4.4 Estado de pruebas

`pytest` ejecuta 16 pruebas en verde: jerarquía PKI y extensiones X.509, política TLS, autenticación con `scrypt` y bloqueo, triggers de inmutabilidad, cadena HMAC de auditoría y la batería de ataques. El proyecto queda completo frente a los objetivos

obligatorios y el extra opcional.